

30 Python Libraries **to (Hugely) Boost** **Your Data Science** **Productivity**

Avi Chawla



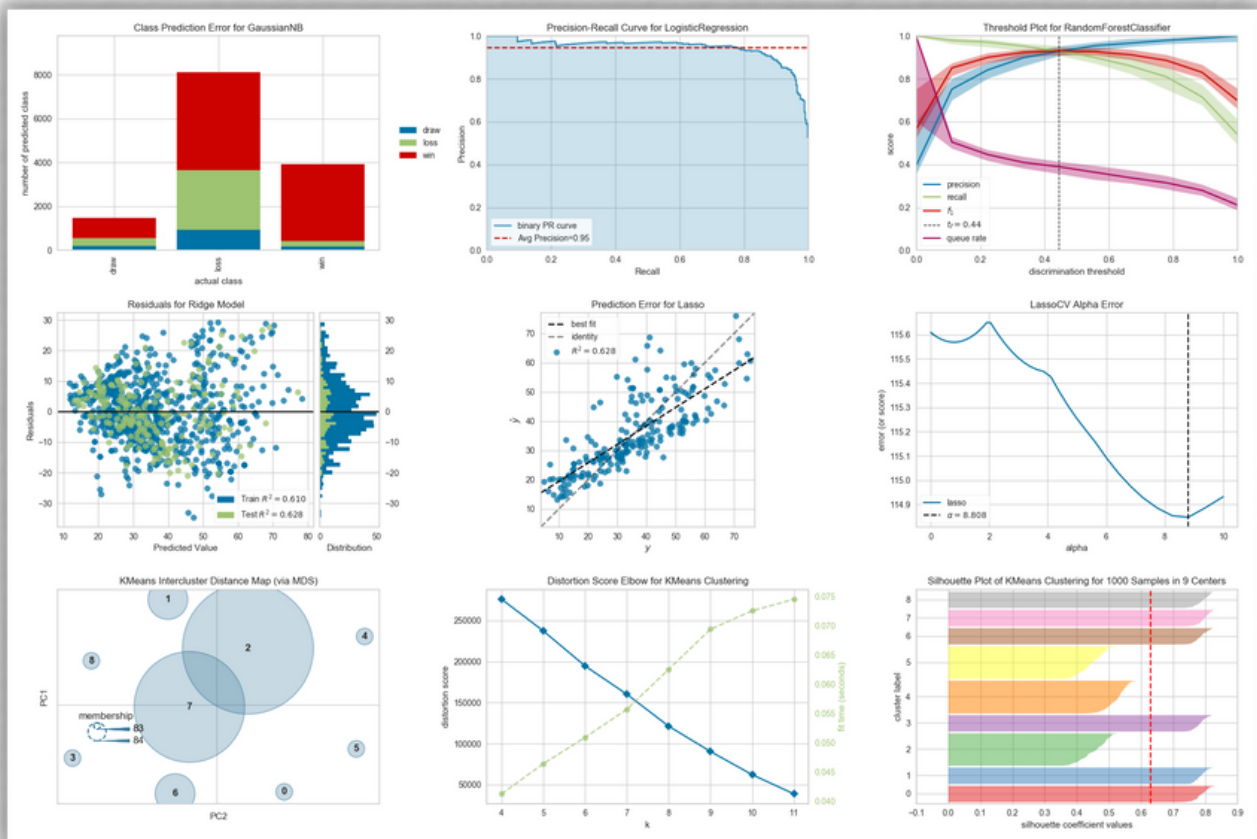
avichawla.substack.com

**Data Science is much
more than Pandas,
NumPy and
Sklearn.**

Here are **30 open-source
libraries** to upgrade
your data game.



1. YellowBrick



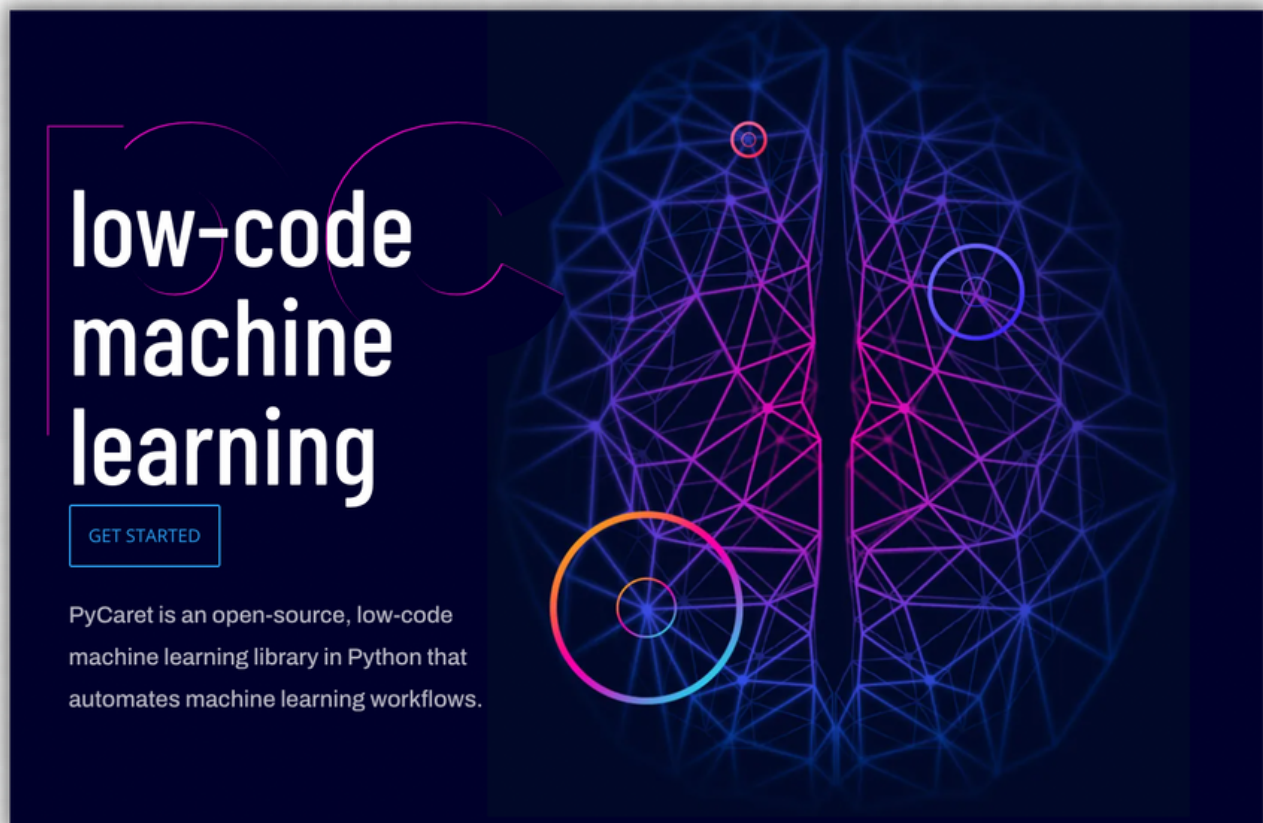
A suite of visualization
and diagnostic tools
for **faster model
selection.**

Matplotlib

Sklearn



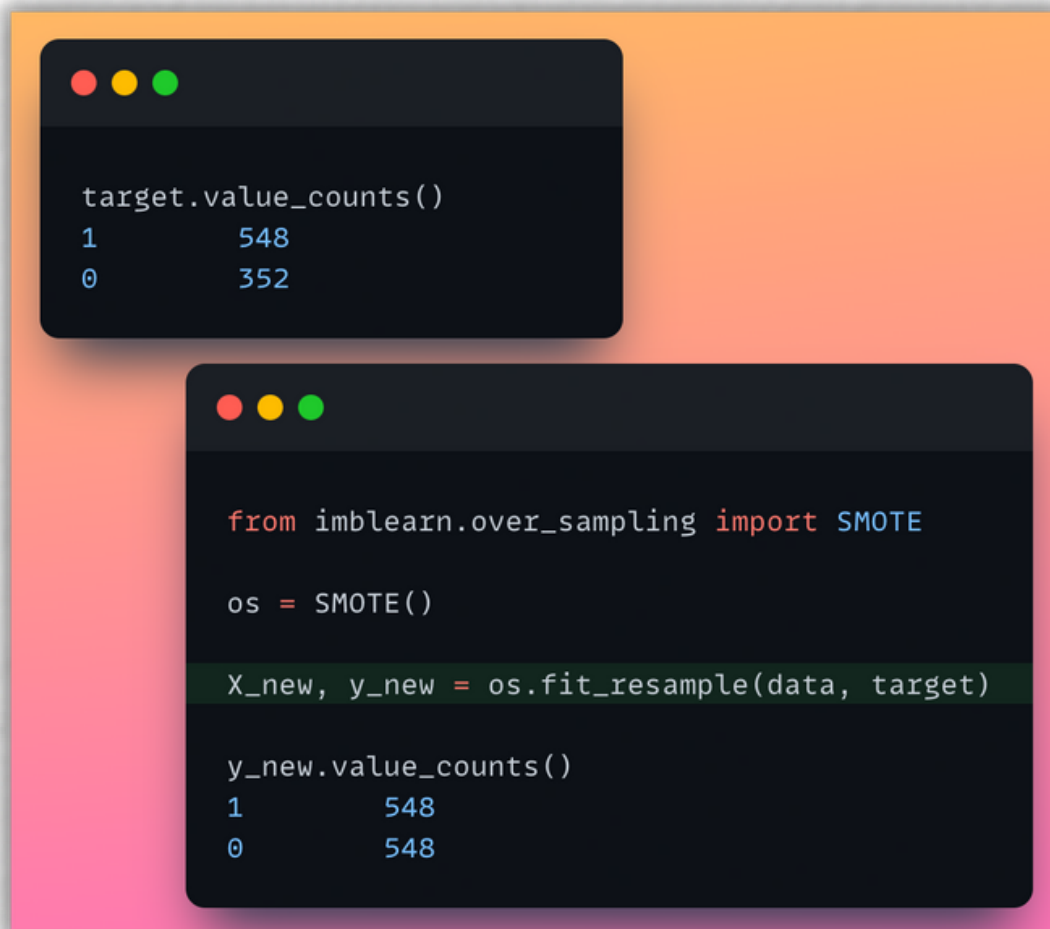
2. PyCaret



Automate ML workflows
with this **low-code
library**.



3. imbalanced-learn



The image shows two terminal windows. The top window displays the output of `target.value_counts()`, showing class 1 with 548 samples and class 0 with 352 samples. The bottom window shows the code to use SMOTE for oversampling class 0, resulting in both classes having 548 samples.

```
target.value_counts()
1      548
0      352
```

```
from imblearn.over_sampling import SMOTE

os = SMOTE()

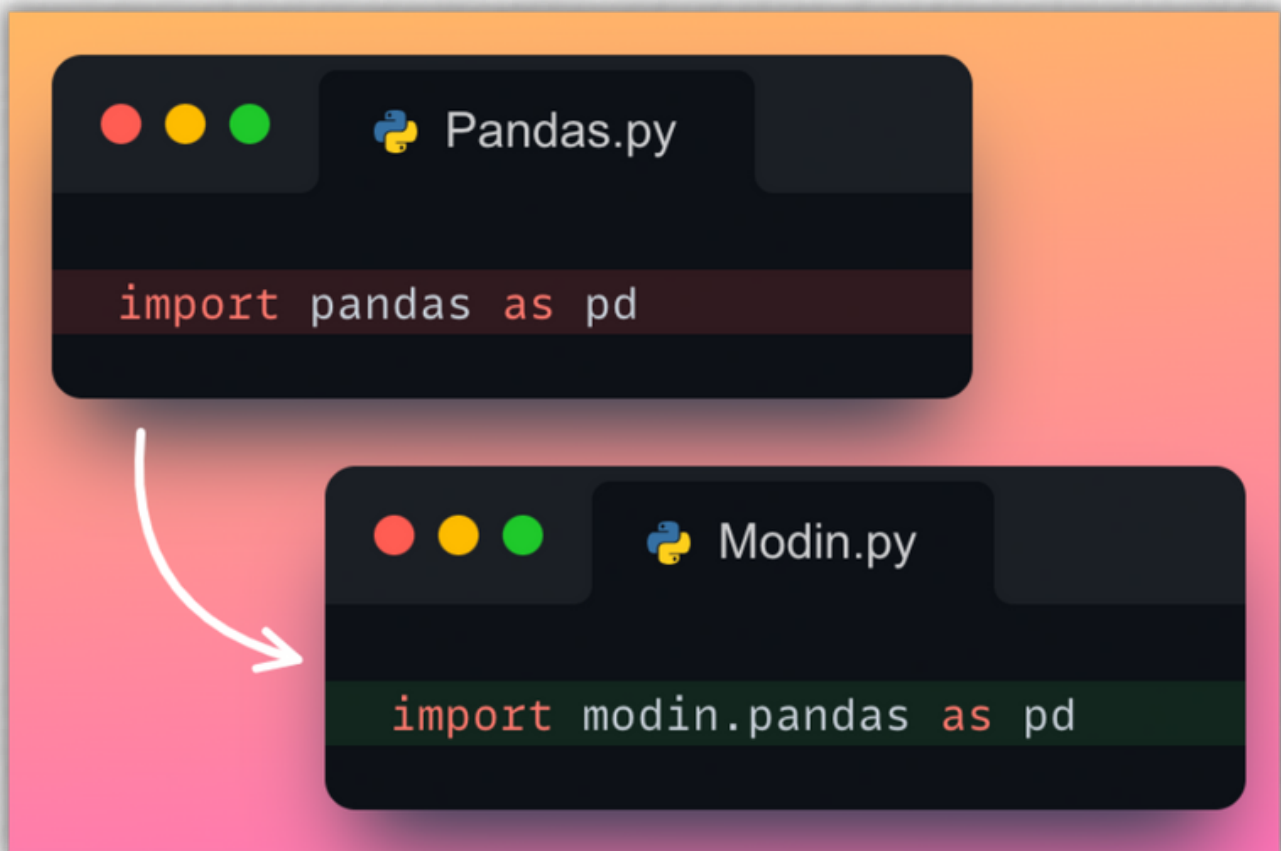
X_new, y_new = os.fit_resample(data, target)

y_new.value_counts()
1      548
0      548
```



A variety of methods
to handle **class**
imbalance.

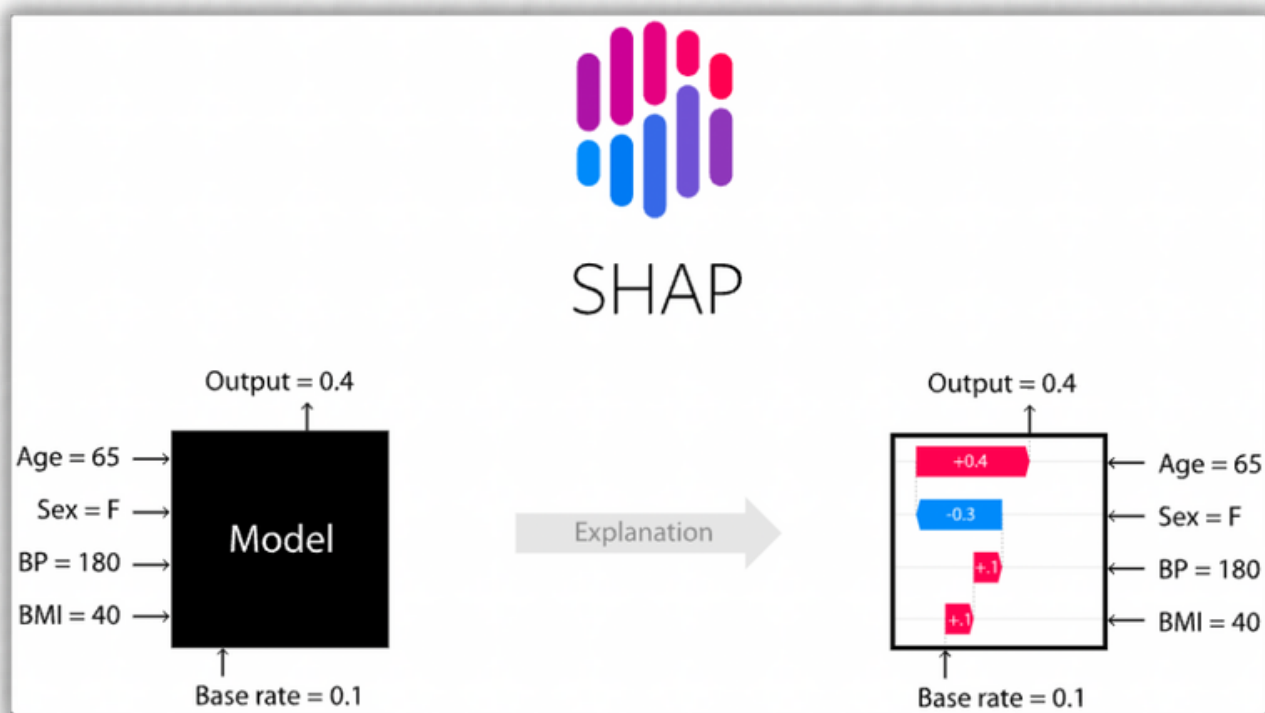
4. Modin



Boost Pandas' performance **up to 70x** by modifying the import.



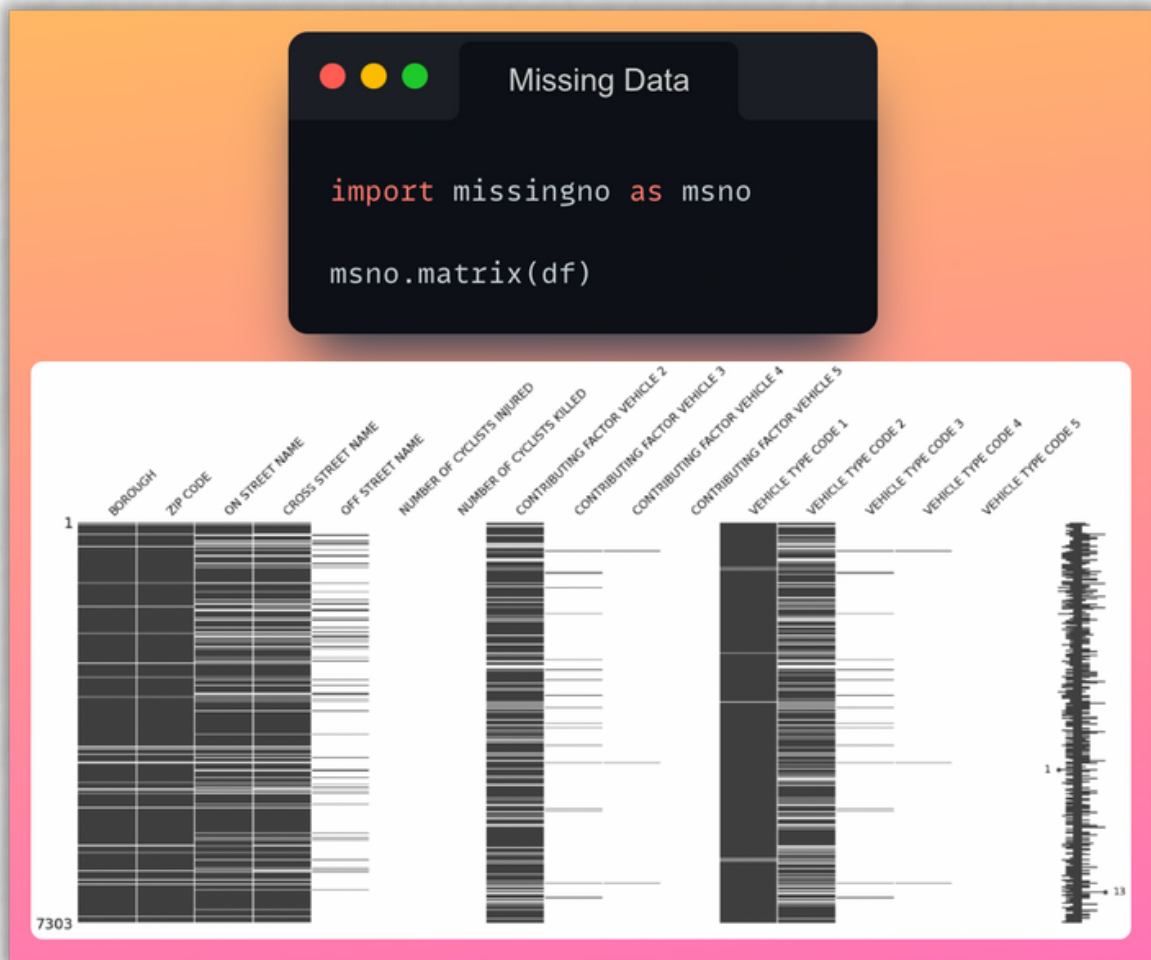
5. SHAP



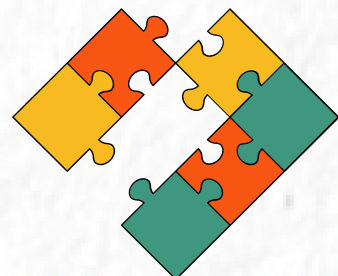
Explain the output of any ML model in few lines of code.



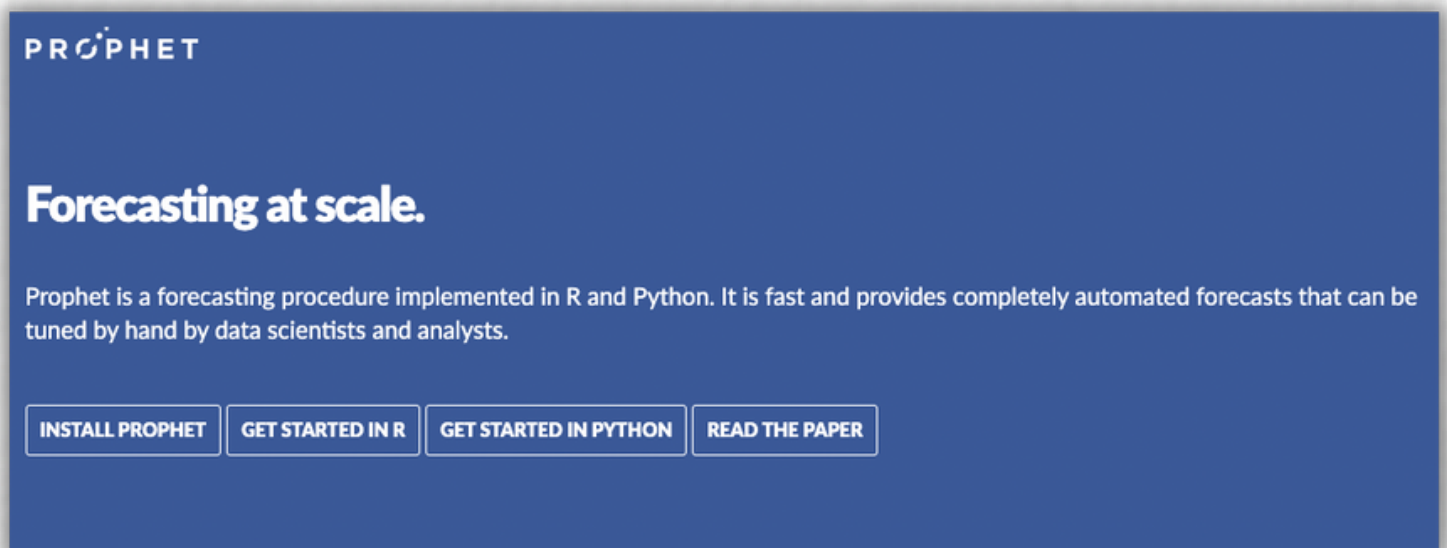
6. Missingno



Visualize missing values
in your dataset
with ease.



7. Prophet



 Produce **high-quality forecasts** on time-series data.

8. Parallel-Pandas

```
from parallel_pandas import ParallelPandas  
  
ParallelPandas.initialize()
```

```
- df.apply()    #Pandas  
+ df.p_apply() #Parallel
```

```
- df.map()      #Pandas  
+ df.p_map()   #Parallel
```

```
- df.mean()     #Pandas  
+ df.p_mean()  #Parallel
```

```
- df.describe() #Pandas  
+ df.p_describe() #Parallel
```

Parallelize Pandas across all CPU cores for **faster computation**.

9. Featuretools



 **Automated feature engineering for ML models.**

10. Lazy Predict

```
from lazypredict.Supervised import LazyRegressor

reg = LazyRegressor()
reg.fit(X_train, X_test, y_train, y_test)
```

```
| Model | R-Squared | RMSE | Time Taken |
|:-----:|:-----:|:-----:|:-----:|
| SVR | 0.877199 | 2.62054 | 0.0330021 |
| KNeighborsRegressor | 0.826307 | 3.1166 | 0.0179954 |
| MLPRegressor | 0.750536 | 3.73503 | 0.725997 |
| ... | ... | ... | ... |
| LinearRegression | 0.71753 | 3.97444 | 0.0190051 |
| DummyRegressor | -0.02157 | 7.55832 | 0.0140116 |
```

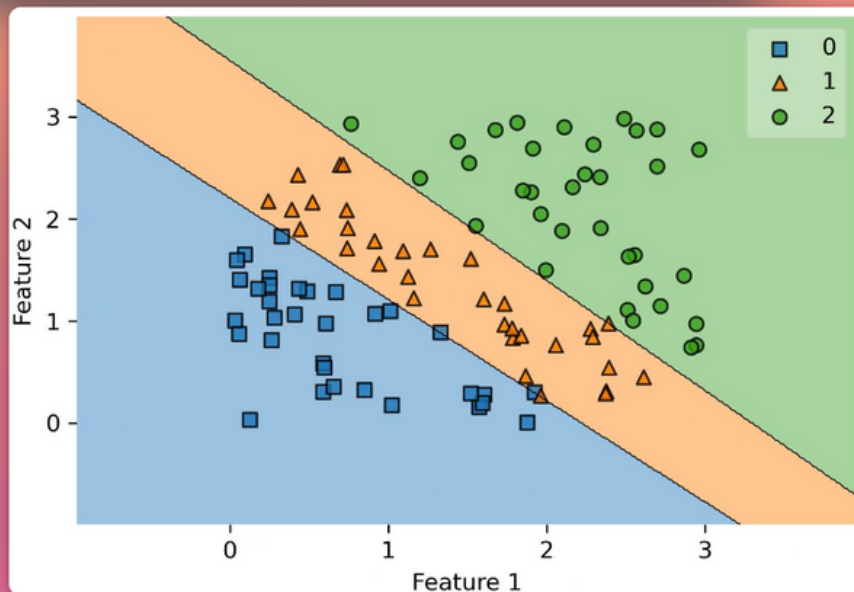
 Train **30 machine learning models** in one line of code.

11. mlxtend

```
from mlxtend.plotting import plot_decision_regions

model = LogisticRegression().fit(X, y)

plot_decision_regions(X, y, model)
```



A collection of utility functions for **processing, evaluating, visualizing** models.

12. Vaex

```
%%time  
df
```

CPU times: user 8 μ s, sys: 0 ns, total: 8 μ s
Wall time: 16.7 μ s

#	vendor_id	pickup_datetime	dropoff_datetime	passenger_count	payment_type	trip_distance	pickup_longitude	pickup_latitude
0	VTs	2009-01-04 02:52:00.000000000	2009-01-04 03:02:00.000000000	1	CASH	2.630000114440918	-73.99195861816406	40.72156524658203
1	VTs	2009-01-04 03:31:00.000000000	2009-01-04 03:38:00.000000000	3	Credit	4.550000190734863	-73.98210144042969	40.736289978027344
2	VTs	2009-01-03 15:43:00.000000000	2009-01-03 15:57:00.000000000	5	Credit	10.350000381469727	-74.0025863647461	40.73974609375
3	DDS	2009-01-01 20:52:58.000000000	2009-01-01 21:14:00.000000000	1	CREDIT	5.0	-73.9742660522461	40.79095458984375
4	DDS	2009-01-24 16:18:23.000000000	2009-01-24 16:24:56.000000000	1	CASH	0.4000000059604645	-74.00157928466797	40.719383239746094
...	...	...	...	...	...	...	...	...
1,173,057,922	VTs	2015-12-31 23:59:56.000000000	2016-01-01 00:08:18.000000000	5	1	1.2000000476837158	-73.99381256103516	40.72087097167969
1,173,057,923	CMT	2015-12-31 23:59:58.000000000	2016-01-01 00:05:19.000000000	2	2	2.0	-73.96527099609375	40.76028060913086
1,173,057,924	CMT	2015-12-31 23:59:59.000000000	2016-01-01 00:12:55.000000000	2	2	3.799999952316284	-73.98729705810547	40.739078521728516
1,173,057,925	VTs	2015-12-31 23:59:59.000000000	2016-01-01 00:10:26.000000000	1	2	1.9600000381469727	-73.99755859375	40.72569274902344
1,173,057,926	VTs	2015-12-31 23:59:59.000000000	2016-01-01 00:21:30.000000000	1	1	1.059999942779541	-73.9843978881836	40.76725769042969



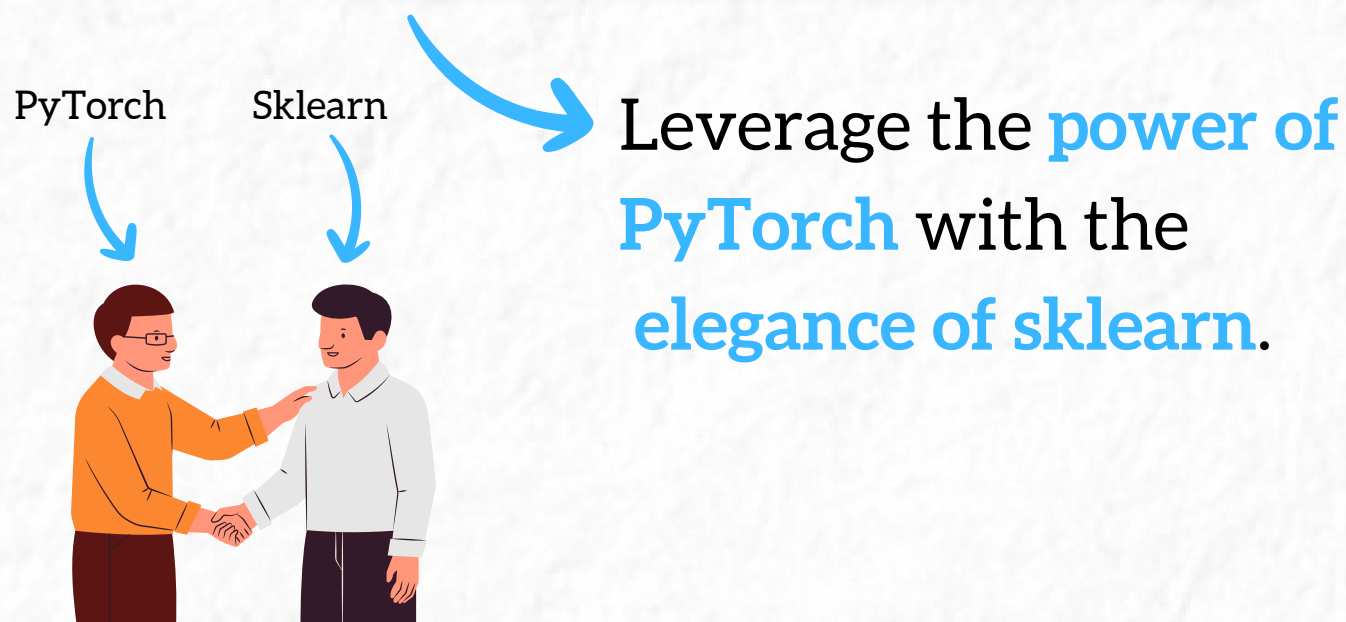
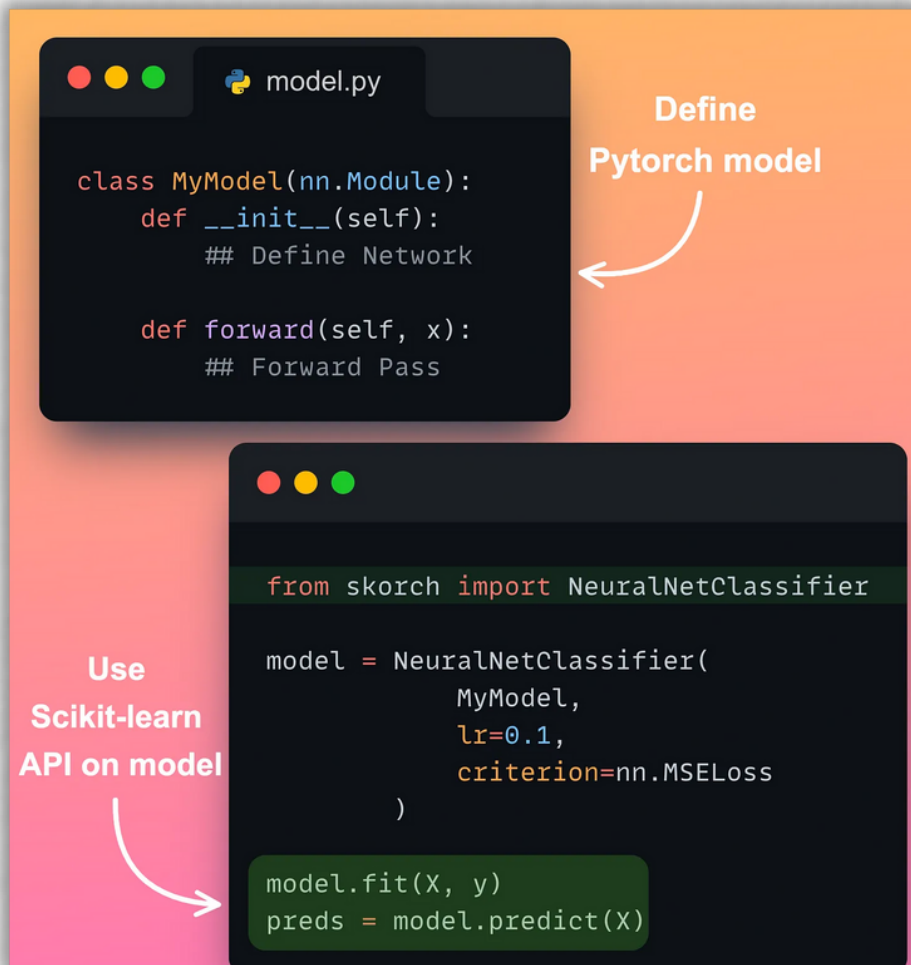
High performance package
for **lazy Out-of-Core
DataFrames.**

13. SweetViz



In-depth EDA report
in two lines
of code.

14. Skorch



15. Faiss



The image shows two code snippets side-by-side, comparing the training time of KMeans using sklearn and faiss. The sklearn snippet shows a training time of 162s, while the faiss snippet shows a training time of 7.8s. A white arrow points from the text '~20x Faster' to the faiss snippet, indicating the performance difference.

```
sklearn.py

from sklearn.cluster import KMeans

kmeans = KMeans(8).fit(x_train)
# Training Time: 162s
```

~20x Faster

```
faiss.py

import faiss

kmeans = faiss.Kmeans(d=1024, k=8)
kmeans.train(x_train)
# Training Time: 7.8s
```

Efficient algorithms for **similarity search** and **clustering** dense vectors.



16. statsmodel

```
import statsmodels.api as sm

results = sm.OLS(y, X).fit()
print(results.summary())
```

OLS Regression Results

Dep. Variable:	y	R-squared:	0.178
Model:	OLS	Adj. R-squared:	0.161
Method:	Least Squares	F-statistic:	10.51
Date:	Wed, 02 Nov 2022	Prob (F-statistic):	7.41e-05
Time:	17:12:45	Log-Likelihood:	-20.926
No. Observations:	100	AIC:	47.85
Df Residuals:	97	BIC:	55.67
Df Model:	2		
Covariance Type:	nonrobust		

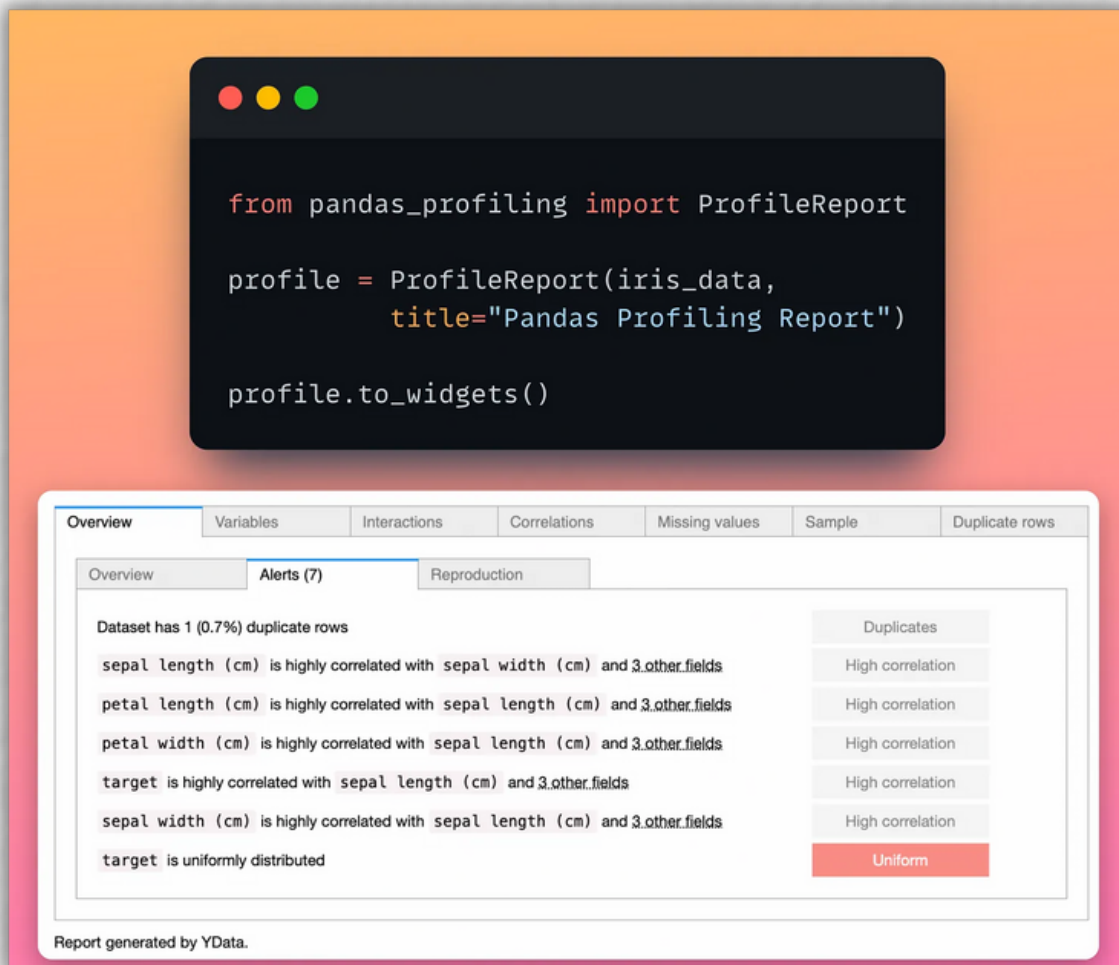
	coef	std err	t	P> t	[0.025	0.975]
const	1.4713	0.075	19.579	0.000	1.322	1.620
x1	0.1045	0.105	1.000	0.320	-0.103	0.312
x2	0.4831	0.107	4.503	0.000	0.270	0.696

Omnibus:	39.684	Durbin-Watson:	1.848
Prob(Omnibus):	0.000	Jarque-Bera (JB):	6.503
Skew:	0.096	Prob(JB):	0.0387
Kurtosis:	1.766	Cond. No.	5.09

Statistical testing and
data exploration
at fingertips.

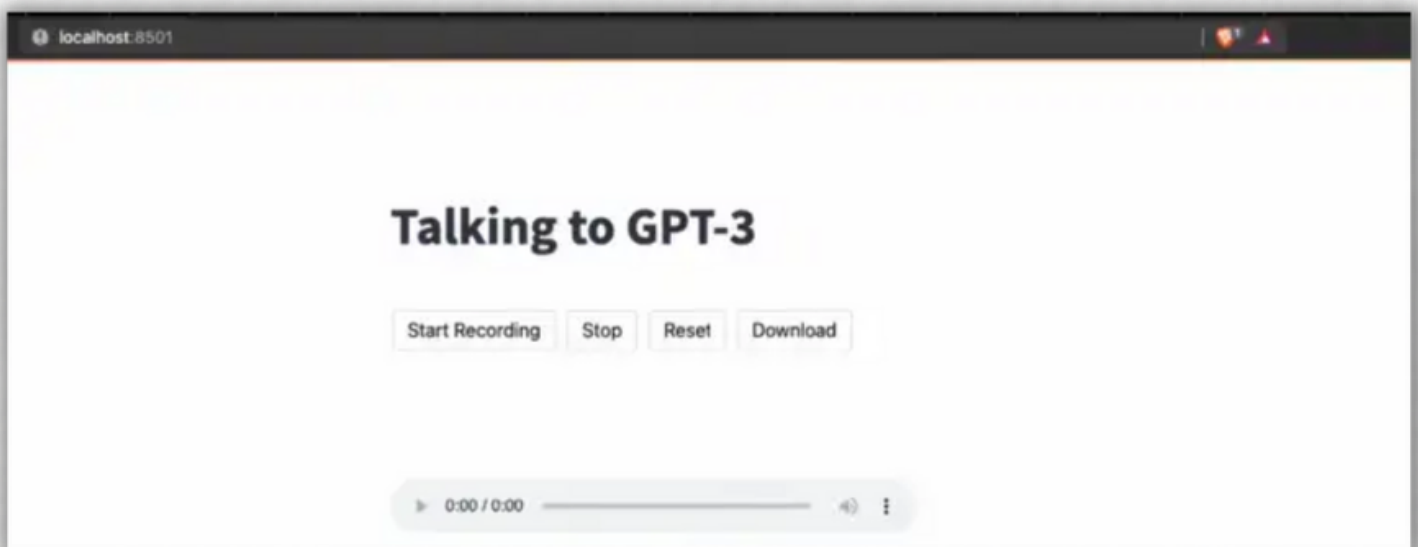


17. Pandas-Profiling



Generate a high-level **EDA report** of your data in no time.

18. Streamlit



 Create and host data-based
Python web apps
in few lines
of code.



19. Category-encoders

```
import category_encoders as ce

ce_be = ce.BinaryEncoder(cols=['class'])

ce_be.fit_transform(data["class"])
```

	class_0	class_1	class_2
0	0	0	1
1	0	1	0
2	0	1	1
3	1	0	0
4	0	0	1

} Binary

	gender	class
0	Male	A
1	Female	B
2	Male	C
3	Female	D
4	Female	A

→ Over 15 categorical data encoders.

20. DuckDB



The diagram illustrates the difference in filtering a DataFrame between Pandas and DuckDB. It features a vertical gradient background transitioning from orange at the top to pink at the bottom. Two code editor windows are shown, each with a title bar containing three colored circles (red, yellow, green) and a filename. The top window, titled 'Filter-Pandas.ipynb', is associated with the 'Pandas' label on the right and contains the code `df[df.city == "New Delhi"]`. The bottom window, titled 'Filter-SQL.ipynb', is associated with the 'DuckDB' label on the left and contains the code `%%sql` followed by a SQL query: `select * from df where city = 'New Delhi';`.

Pandas

```
Filter-Pandas.ipynb
```

```
df[df.city == "New Delhi"]
```

DuckDB

```
Filter-SQL.ipynb
```

```
%%sql
```

```
select * from df
```

```
where city = 'New Delhi';
```

 Run **SQL queries**
on **DataFrame**.

21. PandasML

```
import pandas_ml as pdml

df = pdml.ModelFrame(X, y)

train, test = df.model_selection.
               train_test_split()

reg = df.linear_model.LinearRegression()

train.fit(reg)

test.predict(reg)
```

→ **Pandas** data wrangling +
Sklearn algorithms +
Matplotlib visualization.



22. Pytest



→ An elegant testing framework to test your code.



23. Numexpr

```
import numpy as np
import numexpr as ne
```

```
a = np.random.random(10**7)
b = np.random.random(10**7)
```

```
%timeit np.cos(a) + np.sin(b)
```

142 ms $\pm$ 257 μ s per loop

```
%timeit ne.evaluate("cos(a) + sin(b)")
```

32.5 ms $\pm$ 229 μ s per loop **~5x Faster**

 Parallelize NumPy to
all CPU cores for
20x speedup.



24. CSV-Kit

data.csv

Name	Marks	Grade
Joe	95	A
Hanna	89	B
Chris	92	A
Julie	94	A

Column Names

```
$ csvcut -n data.csv
```

```
1: Name
2: Marks
3: Grade
```

Column Stats

```
$ csvstat data.csv
```

```
2. "Marks"
Type of data:      Number
Contains null values: False
Unique values:     4
Smallest value:    89
Largest value:     95
Sum:               370
Mean:              92.5
Median:            93
StDev:             2.646
```

Query

```
$ csvsql --query "select * from data where Marks>90" data.csv
```

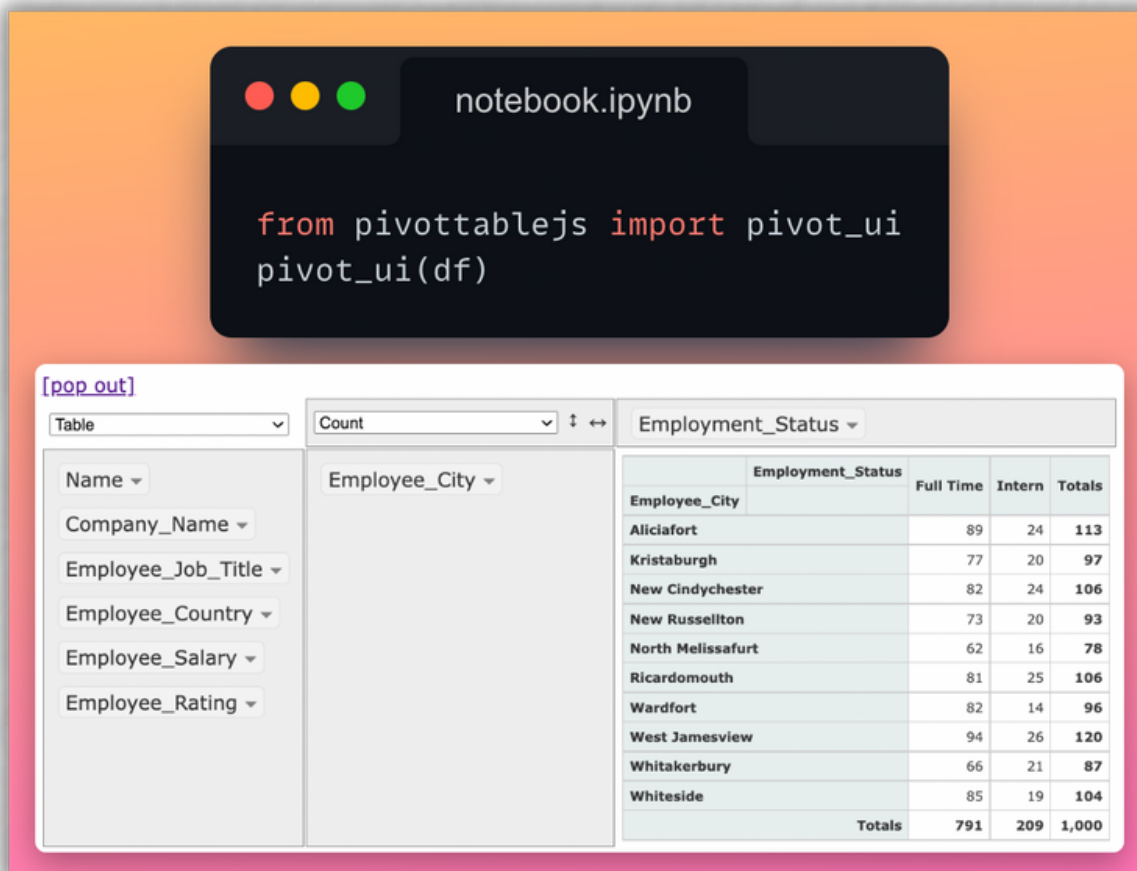
Name	Marks	Grade
Joe	95	A
Chris	92	A
Julie	94	A



Explore, query and describe CSV files from terminal.

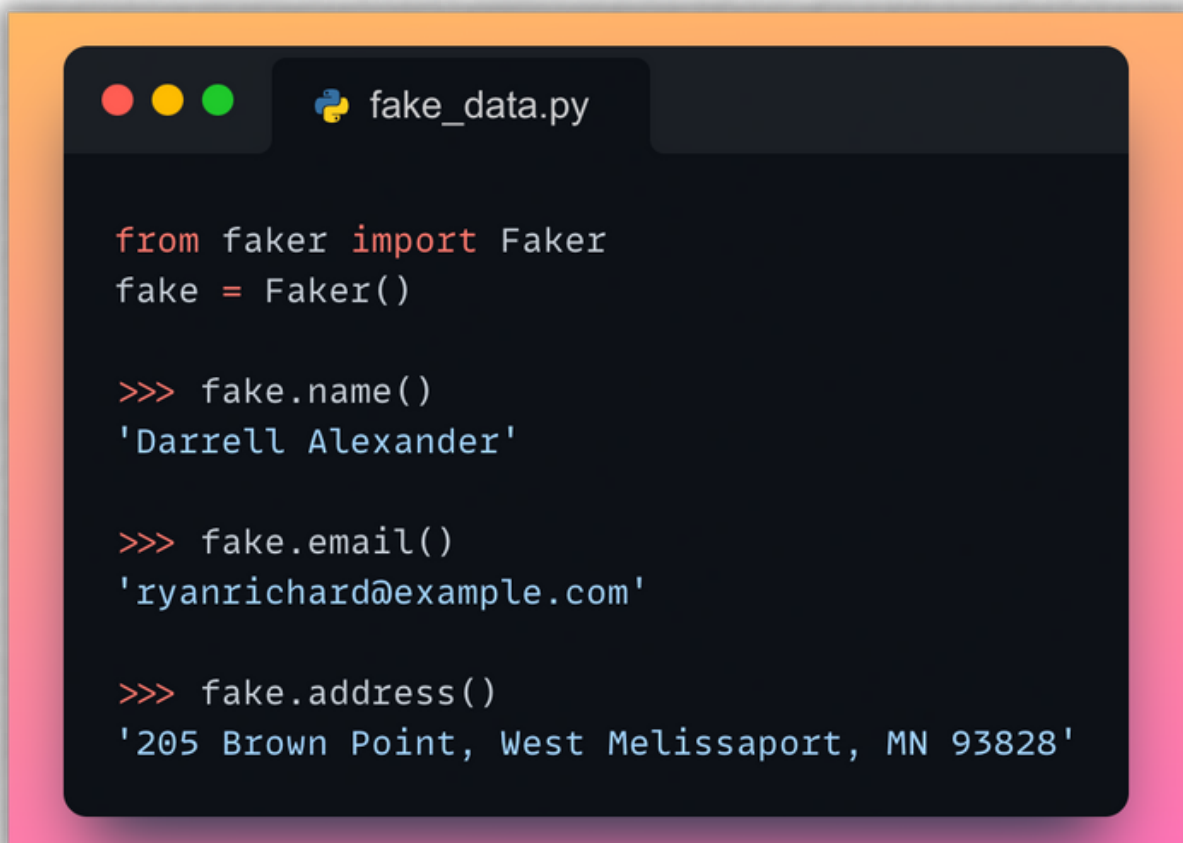
>_

25. PivotTableJS



→ **Drag-n-drop** tools to group, pivot, plot dataframe.

26. Faker



```
from faker import Faker
fake = Faker()

>>> fake.name()
'Darrell Alexander'

>>> fake.email()
'ryanrichard@example.com'

>>> fake.address()
'205 Brown Point, West Melissaport, MN 93828'
```

 Generate **fake yet meaningful data** in seconds.

27. Icecream



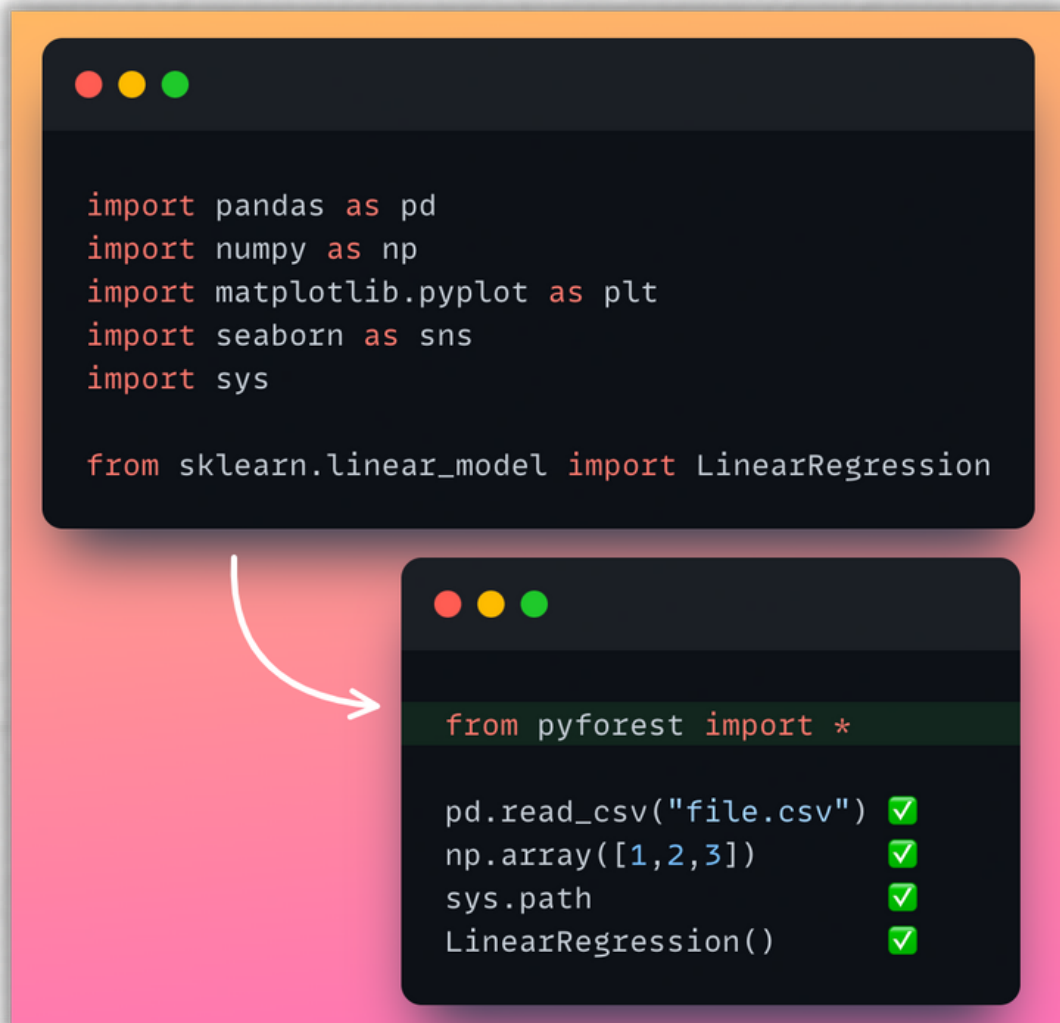
The image shows two code editor windows side-by-side. The top window, titled 'Print', contains Python code that defines a function 'func' which prints its arguments. It then calls 'func([1,2,3], 2)' and the output is shown as a comment: '## arr = [1, 2, 3] n = 2'. The bottom window, titled 'Icecream', shows the same function 'func' but instead of printing, it calls 'ic(arr, n)'. After calling 'func([1,2,3], 2)', the output is shown as a comment: '## ic| arr: [1, 2, 3], n: 2'.

```
def func(arr, n):  
    print("arr =", arr, "n =", n)  
  
func([1,2,3], 2)  
## arr = [1, 2, 3] n = 2
```

```
from icecream import ic  
  
def func(arr, n):  
    ic(arr, n)  
  
func([1,2,3], 2)  
## ic| arr: [1, 2, 3], n: 2
```

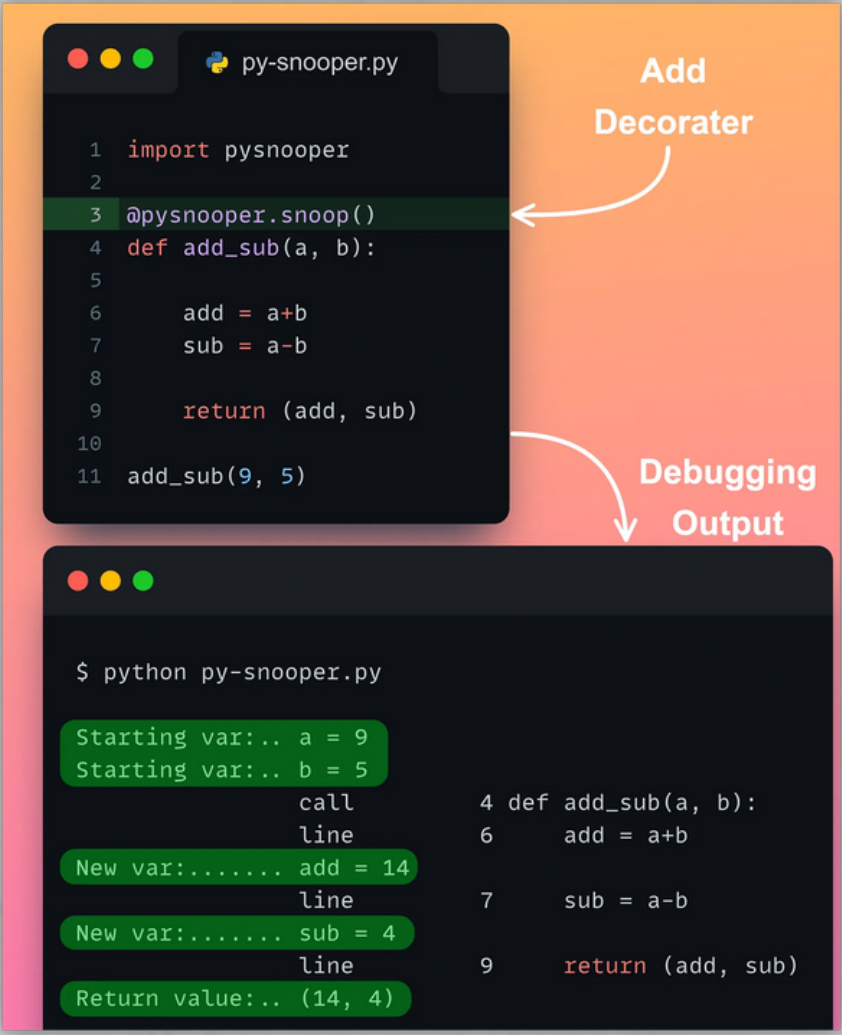
 **Don't debug with `print()`. Use `icecream`.**

28. Pyforest



→ No need to write imports.
**Automatic package
import.**

29. PySnooper



The image shows a code editor window titled 'py-snooper.py' and a terminal window below it. The code editor contains the following Python code:

```
1 import pysnooper
2
3 @pysnooper.snoop()
4 def add_sub(a, b):
5
6     add = a+b
7     sub = a-b
8
9     return (add, sub)
10
11 add_sub(9, 5)
```

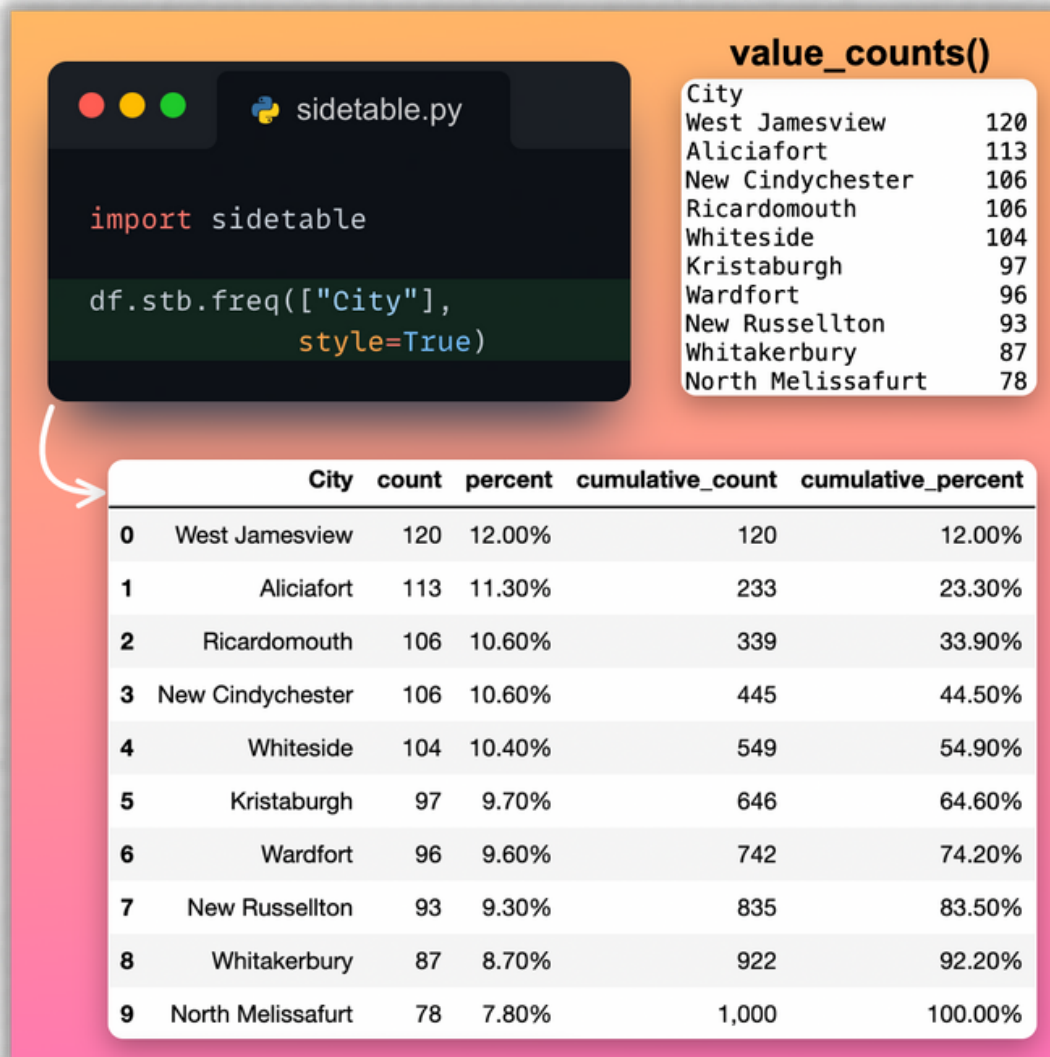
An arrow points from the text 'Add Decorater' to the `@pysnooper.snoop()` decorator on line 3. Another arrow points from the text 'Debugging Output' to the terminal output.

The terminal window shows the command `$ python py-snooper.py` and the following output:

```
Starting var:.. a = 9
Starting var:.. b = 5
      call      4 def add_sub(a, b):
      line      6     add = a+b
New var:..... add = 14
      line      7     sub = a-b
New var:..... sub = 4
      line      9     return (add, sub)
Return value:.. (14, 4)
```

Profile your code. Track new variables, and their updates.

30. Sidetable

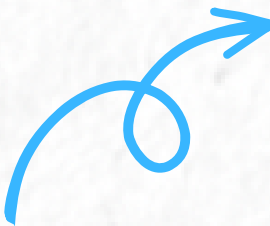


Supercharge Pandas'
value_counts()
method.

Hope that helped.



Checkout my daily newsletter to learn something new about **Python** and **Data Science** everyday.

 avichawla.substack.com

Connect with me on **LinkedIn**.

<https://www.linkedin.com/in/avi-chawla>

